

Un algorithme c'est quoi et comment l'écrire?

Qu'est ce qu'un algorithme?

Quand un problème informatique se présente, on est souvent amené à créer un programme pour le résoudre. Pour créer ce programme là, il faut utiliser un langage de programmation.

Les langages de programmations y'en a des centaines, certains sont très connus et largement utilisés au milieu des programmeurs (comme C, C++, Java, Python...), et d'autres sont moins connus car ils sont trop spécifiques à un domaine en particulier, voire même qu'il sont créés et utilisés uniquement au sein d'une entreprise afin de répondre à la spécificité de ses problèmes. Alors, qui dit beaucoup de langages dit autant de syntaxes. En effet, chaque langage a ses propres caractéristiques, entre autres il y a sa syntaxe. Bien entendu, certains langages de programmation ont parfois des syntaxes proches vu qu'ils sont généralement inspirés les uns des autres. En tout cas, bien qu'il y a autant de syntaxes, tous les programmes créés à l'aide de ces langages partagent un même point commun, il s'agit de la logique de la programmation. Cette logique est tout simplement **l'algorithme**. Par conséquent, un algorithme est indépendant de tout langage de programmation, et en plus, il n'est pas destiné à être exécuté sur l'ordinateur comme c'est le cas d'un programme. En effet, un algorithme peut s'écrire simplement sur du papier ou à l'aide d'un logiciel de traitement de texte et c'est uniquement quand il est jugé efficace qu'il est traduit en programme à l'aide d'un vrai langage de programmation.

Ecrire un algorithme sous forme d'organigramme ou pseudo-code

Souvenez-vous qu'un algorithme n'est pas destiné à être exécuté directement par la machine, mais c'est plutôt son implémentation sous forme de programme qui est destinée à l'exécution sur un ordinateur. Par conséquent, on peut écrire notre algorithme comme on le souhaite, du moment où la personne qui va le traduire en programme pourra le lire et le comprendre. Néanmoins, pour se conformer à ce qui se fait en termes d'algorithmique, deux méthodes sont largement utilisées pour écrire un algorithme: l'organigramme et le pseudo-code.

Les variables et les types

Q'est ce qu'une variable?

Dans un programme informatique, on a souvent besoin de manipuler des valeurs comme les noms des étudiants, les prix de produits, les coefficients d'une équation... Pour stocker ces valeurs on fait appel aux **variables**.

Une variable est une entité dont la valeur peut changer, c'est à dire qu'au sein du même algorithme (ou programme informatique) une même variable peut changer de valeurs de nombreuses fois.

[Un algorithme](#) tourne généralement autour des variables. C'est à dire que c'est derniers sont les éléments les plus importants et toutes les instructions de

l'algorithme ont pour mission de leur affecter des valeurs calculées avec précision. A la fin, ce sont ces valeurs-là qui seront rendues à l'utilisateur après l'exécution.

Les variables doivent avoir un type

Dans la plupart des langages de programmation, avant de manipuler une variable, il faut préalablement déclarer son type. C'est à dire que la variable en question ne pourra changer de valeur que dans l'intervalle défini par le type qui lui est assigné.

Dans un algorithme, on se contente de 4 type de base, à savoir:

- **Les entiers:** qui sont des nombres sans virgule et qui peuvent être positifs ou négatifs. On parle alors de [nombres entiers signés](#).
- **Les réels:** qui sont des nombres avec virgule (dite [virgule flottante](#)) et qui peuvent être positifs ou négatifs aussi
- **Les booléens:** qui définissent deux valeurs (dites binaires) qui sont Vrai ou Faux (ou encore 1 ou 0)
- **Les chaînes de caractères:** qui représentent des textes constitués de tout type de caractères comme les caractères alphabétique, numériques et symboles.

Les opérateurs

A quoi servent les opérateurs?

Les opérateurs sont des symboles qui permettent d'exécuter des opérations dans [un algorithme](#) ou programme informatique. Par exemple si on veut effectuer une addition de deux variables entiers ou réels on utilise l'opérateur d'addition connu par le célèbre symbole +.

En algorithmique, on dénombre généralement 4 familles d'opérateurs:

- **Opérateurs d'assignation (ou affectation):** Ce sont des opérateurs qui permettent d'affecter (assigner) une valeur à une variables.
- **Opérateurs de [calcul](#) (ou arithmétiques):** Ce sont des opérateurs qui permettent d'effectuer des opérations de calcul comme l'addition ou le produit.
- **Opérateurs de comparaison:** Ce sont des opérateurs qui permettent de comparer deux éléments (variables, constantes ou valeurs statiques).
- **Opérateurs logiques:** Ce sont des opérateurs qui permettent d'effectuer des opérations logiques comme le ET logique ou le OU logique...

La lecture et l'écriture

Rendre le programme (algorithme) interactif

Quand un programme informatique qui implémente [l'algorithme](#) en question s'exécute, alors il doit impérativement faire parvenir l'utilisateur (c'est à dire la personne qui exécute le programme) de son état. Autrement dit, le programme doit

afficher à l'écran des messages qui indiquent ce qu'il est entrain de faire. En d'autre part, l'utilisateur doit, de temps à autre, renseigner au programme des valeurs dont il a besoin pour poursuivre son exécution. On parle alors d'interaction homme-machine, c'est à dire que le programme a la capacité d'interagir avec l'utilisateur (ou son environnement d'exécution en général) en accueillant des données à l'entrée et en retournant des résultats à la sortie. L'entrée peut être le clavier qui permet par exemple à l'utilisateur d'assigner une valeur à une variable et la sortie peut être l'écran où le programme affiche des messages que l'utilisateur peut lire (comme le résultat d'addition de deux nombres dont les valeurs ont été initialisées par ce dernier). Afin de mettre en place cette interaction, en algorithmique on dispose des fonctions **lire()** et **écrire()**.

La fonction lire() permet au programme d'accueillir des valeurs saisies par l'utilisateur, et la fonction écrire() permet d'afficher des messages (à l'écran par exemple) afin que l'utilisateur puisse les lire.

Les chaînes de caractères

Qu'est ce qu'une chaîne de caractères?

Une **chaîne de caractères** est un type que l'on peut donner à une [variable](#) dans [un algorithme](#) (ou programme informatique). La chaîne de caractères est sans doute le type de variables le plus populaire et le plus utilisé dans de nombreux langages de programmation. Certains langages en ont même fait leur type vedette comme c'est le cas du [langage Perl](#).

Une chaîne de caractère est un texte. La valeur que l'on peut affecter à une variable de type chaîne de caractères peut être un mot, une phrase ou un texte composé de plusieurs paragraphes. Donc dans une chaîne on peut trouver des caractères alphabétiques (accentués ou non), des caractères numériques, les marqueurs de ponctuation, les caractères spéciaux... On peut même trouver des caractères provenant d'autres systèmes d'écriture comme le grec ou l'arabe...

L'implémentation des chaînes de caractères dans les langages de programmation est étroitement lié à au système d'encodage comme l'ASCII, Unicode... Mais en algorithmique on ne s'intéresse pas à ces détails techniques et on manipule les variables de types chaînes de caractères normalement.

Les conditions

La structure conditionnelle Si - Sinon

Un programme informatique qui implémente [un algorithme](#) donné n'évolue pas toujours de la même manière à chaque exécution. Il se trouve des fois que face à certaines circonstances le programme change de comportement et c'est souvent le cas dans les vrais programmes.

En réalité, bien que le code source d'un programme (ou algorithme) est le même, mais les résultats de son exécution changent de temps à autre et c'est souvent

l'oeuvre des **structures conditionnelles** (ou **conditions**).

Imaginons que l'on veut écrire un algorithme qui permet de calculer le résultat de la division de deux nombres. Si le dénominateur n'est pas nul, alors l'opération de division est faite et le résultat est affiché à l'écran. Par contre, si le dénominateur est nul, dans ce cas l'opération de division ne peut pas être faite et le programme nous affiche à l'écran un message d'erreur par exemple. Là, on a bien un cas où le traitement conditionnel est requis.

En pseudo-code on exprime les structures conditionnelles à l'aide des mots-clés **Si**, **Sinon** et **FinSi**.

Structure sélective (ou structure conditionnelle à choix multiples)

Structure Selon (ou Cas)

La **structure sélective** ou **structure conditionnelle à choix multiples** ou encore **structure Selon** (appelée parfois structure Cas) est une représentation simplifiée des [conditions imbriquées \(Si - Sinon\)](#). En effet, quand on a affaire à beaucoup de conditions à traiter dans un algorithme (ou programme informatique), alors la structure sélective semble la solution la plus appropriée car elle est plus facile à représenter (au lieu d'exprimer beaucoup d'imbrications de conditions Si - Sinon).

La boucle TantQue

Les structures itératives

Dans [un algorithme](#) (ou [programme informatique](#)) on a souvent besoin de répéter un même bloc d'instructions plusieurs fois. Au lieu d'effectuer cette répétition manuellement on utilise les **structures itératives**.

En algorithmique on dénombre généralement 3 structures itératives à savoir, **la boucle TantQue**, [la boucle Pour](#) et [la boucle Répéter](#).

La structure TantQue, dite aussi **boucle TantQue**, exécute un même bloc d'instructions tant que [la condition](#) spécifiée dans l'algorithme est vraie. Une fois cette condition là devient fausse, alors on quitte la boucle pour poursuivre l'exécution du reste du traitement.

En général, on préfère la boucle TantQue si le nombre d'itérations à exécuter n'est pas connu à l'avance. Cependant, on peut exécuter le même traitement itératif à l'aide de n'importe laquelle des 3 boucles connues.

La boucle Pour

Une structure itérative plus simple à déclarer

Dans [un algorithme](#) (ou programme informatique) on a souvent besoin de répéter un même bloc d'instructions de nombreuses fois et de manière automatique. C'est justement le rôle des structures itératives.

Dans la leçon consacrée à [la boucle TantQue](#), nous avons vu que cette structure itérative est utile quand on ne connaît pas à l'avance le nombre d'itérations à

exécuter. Donc, la boucle s'exécute tant que la condition prévue est vraie. Pourtant, dans une telle structure on a souvent besoin de déclarer les 3 instructions de base suivantes:

- L'initiation du compteur de la boucle
- Spécification de [la condition](#) qui permet de quitter la boucle
- Modification de la valeur du compteur de la boucle (par incrémentation ou décrémentation par exemple)

La boucle Pour permet de déclarer ces trois instructions dans une même ligne et de manière très simple. Cette simplicité confère à la boucle Pour une grande popularité, ce qui la place à la tête des structures itératives les plus utilisées par les programmeurs. En revanche, la boucle Pour est utile quand on connaît à l'avance le nombre d'itérations à exécuter. Si ce nombre là est incertain alors la boucle Pour ne constitue pas la meilleur structure itérative à implémenter.

La boucle Répéter

Une première itération exécutée dans tous les cas

Dans les leçons où j'ai traité [la boucle TantQue](#) et [la boucle Pour](#), j'ai mentionné que ces deux boucles sont relativement similaires et permettent d'exécuter le même bloc d'instructions plusieurs fois dans [un algorithme](#) (ou programme informatique), sauf qu'on préfère utiliser la boucle Pour si l'on connaît à l'avance le nombre d'itérations à exécuter et la boucle TantQue dans le cas contraire. Cependant, dans ces deux structures, si jamais [la condition](#) de sortie de la boucle est satisfaite depuis le début alors aucune itération ne sera exécutée ni dans la boucle TantQue, ni dans la boucle Pour.

La boucle Répéter quant-à elle est aussi une structure itérative, mais sa première itération est toujours exécutée quelque soit la condition de sortie de la boucle. En effet, la vérification de la condition n'est faite qu'après avoir exécuté le bloc d'instructions.

La boucle Répéter est souvent préférée si l'on souhaite filtrer des entrées de l'utilisateur, ou exécuter toute autre opération qui requière au moins une itération obligatoire.

Les tableaux

Une variable de type tableau

Les tableaux dans [un algorithme](#) (ou programme informatique) sont [des variables](#) qui peuvent contenir plusieurs valeurs simultanément. En effet, les variables vues jusqu'ici (entiers, réels, booléens et chaînes de caractères) sont dites des variables scalaires. Il s'agit de variables qui peuvent accueillir une seule valeur à la fois. Le fait de leur affecter une nouvelle valeur écrase systématiquement l'ancienne.

Dans les tableaux on peut stocker plusieurs valeurs simultanément. En effet, un tableau est vu comme une structure organisée sous forme de cellules (ou cases). Chaque case a un numéro qui permet de l'identifier. Ce numéro est appelé indice (ou index).

En algorithmique, les tableaux peuvent contenir des valeurs du même type. On parle alors d'un tableau d'entiers, un tableau de chaînes de caractères etc...

Afin de connaître le nombre d'éléments inclus dans un tableau on utilise la fonction **longueur()**.

Les sous-programmes: Fonctions et procédures

Pourquoi les sous-programmes?

Dans [un algorithme](#) (ou programme informatique) on se retrouve souvent avec des bouts de code qui peuvent se répéter dans plusieurs emplacement. Cette redondance rend le code long, moins lisible et, par dessus tout, si on souhaite modifier le comportement de ces blocs redondants alors on doit les réécrire tous, ce qui n'est pas du tout pratique.

Les sous-programmes permettent d'éviter cette redondance en factorisant le code qui peut se répéter. De cette façon nous n'aurons qu'un seul bloc de code que l'on peut réutiliser à volonté dans plusieurs endroit sans être obligé de le réécrire.

Les sous-programmes peuvent être **des fonctions** ou **des procédures**.

Les fonctions et procédures sont presque similaires, sauf qu'une fonction retourne un résultat au moment de son appel, alors qu'une procédure ne retourne rien.

Cependant, les fonctions et les procédures ont un nom, un bloc pour déclarer [les variables](#) (qui ont souvent une portée locale), un corps principal et peuvent avoir des arguments.

La récursivité

Les fonctions récursives

Une fonction récursive est une [fonction](#) qui s'appelle elle même. **La récursivité** produit une sorte d'itération identique à celle produite par les boucles que nous avons déjà traitées, à savoir, [la boucle TantQue](#), [la boucle Pour](#) et [la boucle Répéter](#).

Dans certains problèmes informatiques qui requièrent un traitement itératif, la récursivité semble être la meilleure solution. En effet, la récursivité donne lieu à un code plus lisible et en plus, le temps d'exécution du programme qui implémente l'algorithme en question se trouve réduit. C'est souvent le cas dans les algorithmes de tri par exemple.

Notez qu'une récursivité mal formulée peut donner lieu à un traitement infini exactement comme une boucle infinie dans le cas des structures itératives mal écrites.

La complexité des algorithmes

Calcul de la complexité asymptotique

L'efficacité d'un [algorithme](#) est jugée par l'évaluation de son temps d'exécution et par les ressources matérielles mises à sa disposition au moment de l'exécution.

Cependant, pour évaluer l'efficacité d'un algorithme on s'intéresse surtout à calculer son temps d'exécution. Mais vu que ce temps peut changer d'une machine à une autre (une machine plus performante exécutera l'algorithme plus rapidement), alors on s'intéresse surtout à évaluer l'ordre de grandeur du nombre d'opérations exécutées dans un algorithme. On parle alors de **complexité asymptotique**.

La complexité asymptotique est représentée par le symbole **Grand O**. On peut donc avoir une complexité constante qui vaut $O(1)$, une complexité linéaire qui vaut $O(n)$, une complexité quadratique qui vaut $O(n^2)$, une complexité logarithmique qui vaut $O(\log(n))$ etc...

Équation du deuxième degré

Écrire un algorithme qui permet de résoudre l'équation du second degré

Dans cet exercice corrigé nous allons traiter un classique de la programmation pour débutants. Il s'agit d'écrire un algorithme qui permet de résoudre l'équation du deuxième degré (ou équation du second degré) qui a la forme $ax^2+bx+c=0$.

La méthode consiste à calculer le discriminant (Delta), ensuite on évalue le signe de celui-ci pour en déduire les solutions possibles. Le traitement principal dans l'algorithme consiste à l'imbrication des conditions (ou structures conditionnelles imbriquées) en utilisant les mots-clés Si Alors Sinon et Finsi. Quant-aux coefficients de l'équation, ils seront saisis par l'utilisateur.